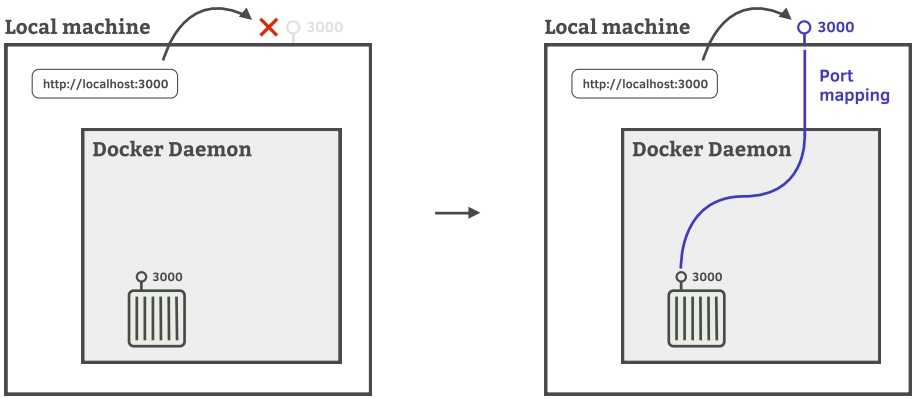
The truth is, containers wouldn't be very useful if there was no way of reaching them from outside of Docker. The whole point of a web server, for example, is that it's accessible to people making requests.

Although, by default, a container can only be accessed from within the Docker network it's connected to (more on this in How Containers Can Talk to Each Other, on page 63), we can make it accessible externally by *publishing one or more ports* with `docker run`'s `-p` option.

In our command, we specified `-p 3000:3000`; this publishes the container's port 3000 (the second number) on port 3000 on our local machine. That means that a request to http://localhost:3000 will reach our Rails server running inside the container on port 3000.

How does this work in practice?

As we saw in the Docker for Linux architecture diagram on page 14, on Linux, the Docker daemon runs directly on the local machine. In this case, publishing a port simply sets up a port mapping (via an `iptables` rule), which forwards requests to http://localhost:3000 on to the Docker Engine, which knows to route the request to the network the container is on (as shown in the following figure).



`docker run ...` `docker run -p 3000:3000 ...`

Docker for Mac/Windows has an added complication. Remember, here the Docker daemon is running inside a lightweight Linux VM, as we saw in the Docker for Mac/Windows architecture diagram on page 14. Inside the VM, things work exactly as Docker on Linux—the port mapping will route requests to the container. However, an extra bit of magic is needed to forward requests from http://localhost:3000 to the VM's port 3000; Docker for Mac/Linux sets up port forwarding to achieve this, as illustrated in the figure on page 28.